



High-Level Overview of Compilation

What is compilation?

Compilation is the process of converting human-readable source code into machine-executable instructions. It involves several steps:

1. Reading source files
2. Analyzing the code
3. Linking different parts of the program
4. Optimizing the code
5. Generating executable code

The three main phases we focus on are preprocessing, object generation and linking

Preprocessing

- Preprocessing occurs before the main compilation.
- It handles directives that start with '#' in C.
- These directives instruct the compiler to perform certain actions before compiling the main code.

Pre-compiler Directives

- Common precompiler directives include:
- `#include`: Inserts the contents of another file
- `#define`: Creates macros or constants
- Macro Expansion: Replaces macros with their defined content
- `#ifdef/#ifndef/#endif`: Conditional compilation
- `#pragma`: Gives special instructions to the compiler

Macro Expansion

Macros are like text substitutions. For example:

```
# define min(x,y) ((x)<(y)?(x):(y))
```

This defines a macro that finds the minimum of two values.

Note:

- Macros can be tricky and may cause unexpected behavior.
- It's often better to use functions instead.

Compilation and Object Files

In larger project with multiple source files, each is compiled separately into an object file (*.o). Object files contain machine code but are not yet complete executables. They have:

- Executable code
- Unresolved links to other parts of the program
- No defined entry point (like `main()`)

Linking

The linker's job is to combine all the object files and resolve the links between them.

It does the following:

1. Connects the function calls to their definitions
2. Resolves references to external libraries
3. Creates the final executable with a defined entry point

This process allows for modular development, where different parts of a program can be compiled separately and then linked together to form a complete application.